

AWS LoadBalancers

1. Configure Classic Load balancer.

Objective: To configure an AWS Classic Load Balancer (CLB) that distributes incoming HTTP traffic across multiple EC2 instances to improve application availability and fault tolerance.

Prerequisites

Before creating the Classic Load Balancer, ensure:

- Two EC2 instances are running.
- Both instances are in the same VPC.
- Both instances are in different Availability Zones (recommended).
- Apache or Nginx is installed and running.
- Security Groups allow HTTP (Port 80).
- Health check page is accessible.

Step 1: Login to AWS Console

1. Login to AWS Management Console.
2. Select your AWS Region.
3. Go to

EC2

Step 2: Open Load Balancers

Left Panel

Load Balancing



Load Balancers

Click

Create Load Balancer

Step 3: Choose Load Balancer Type

AWS shows three options

- Application Load Balancer
- Network Load Balancer
- Classic Load Balancer

Select

Classic Load Balancer

Click

Create

Step 4: Configure Basic Details

Enter

Name:

My-Classic-LB

Scheme

Internet-facing

Network Mapping

Choose your VPC

Example

Default VPC

Availability Zones

Select

us-east-1a

us-east-1b

(Choose AZs where your EC2 instances are running.)

Click

Next

The screenshot shows the AWS Management Console interface for creating a Classic Load Balancer. The breadcrumb navigation at the top indicates the path: EC2 > Load balancers > Create Classic Load Balancer. On the left, there is a navigation menu with categories like EC2, Instances, Images, Elastic Block Store, and Network & Security. The main content area is titled 'Create Classic Load Balancer' and is divided into two main sections: 'Basic configuration' and 'Network mapping'. In the 'Basic configuration' section, the 'Load balancer name' is set to 'My-Classic-LB'. Below this, the 'Scheme' is set to 'Internet-facing', which is highlighted with a blue border. The 'Internet-facing' scheme is described as serving internet-facing traffic, having public IP addresses, resolving DNS names to public IPs, and requiring a public subnet. The 'Internal' scheme is also visible but not selected. In the 'Network mapping' section, the 'VPC' is set to 'vpc-083187674976c0db2'. A 'Create VPC' button is located to the right of the VPC dropdown. The page also includes a 'Learn more' link and a 'default' dropdown menu.

Step 5: Configure Listener

Default Listener

Protocol: HTTP

Load Balancer Port: 80

Instance Protocol: HTTP

Instance Port: 80

Click

Next

EC2 > Load balancers > Create Classic Load Balancer

EC2

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Listeners and routing

A listener is a process that checks for connection requests using the protocol and port you configure. The settings you define for a listener determine how the load balancer routes requests to its registered targets.

Listener HTTP:80

Remove

Instance HTTP:80

Listener protocol

HTTP

Instance protocol

HTTP

Listener port

80

Instance port

80

1-65535

1-65535

Add listener

You can add up to 99 more listeners.

Step 6: Configure Security Group

Choose

Existing Security Group

or

Create New Security Group

Inbound Rules

Type	Protocol	Port	Source
------	----------	------	--------

HTTP	TCP	80	0.0.0.0/0
------	-----	----	-----------

Click

Next

Step 7: Configure Health Check

Health Check Settings

Ping Protocol:

HTTP

Ping Port:

80

Ping Path: /index.html

Step 8: Add EC2 Instances

AWS displays running instances.

Select: server-1 and server-2

Step 9: Review

Review all settings.

Click

Create

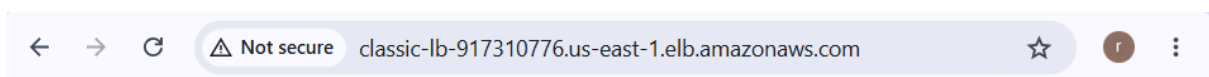
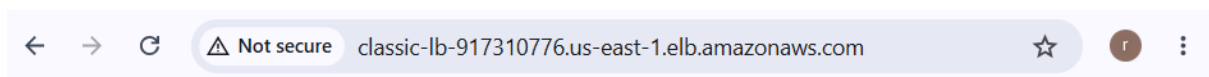
Classic Load Balancer is created.

Step 10: Copy DNS Name

Example: Classic-LB-917310776.us-east-1.elb.amazonaws.com

Open browser: http://DNS-Name

Your application should load.



Step 12: Verify Health Check

Go to

EC2

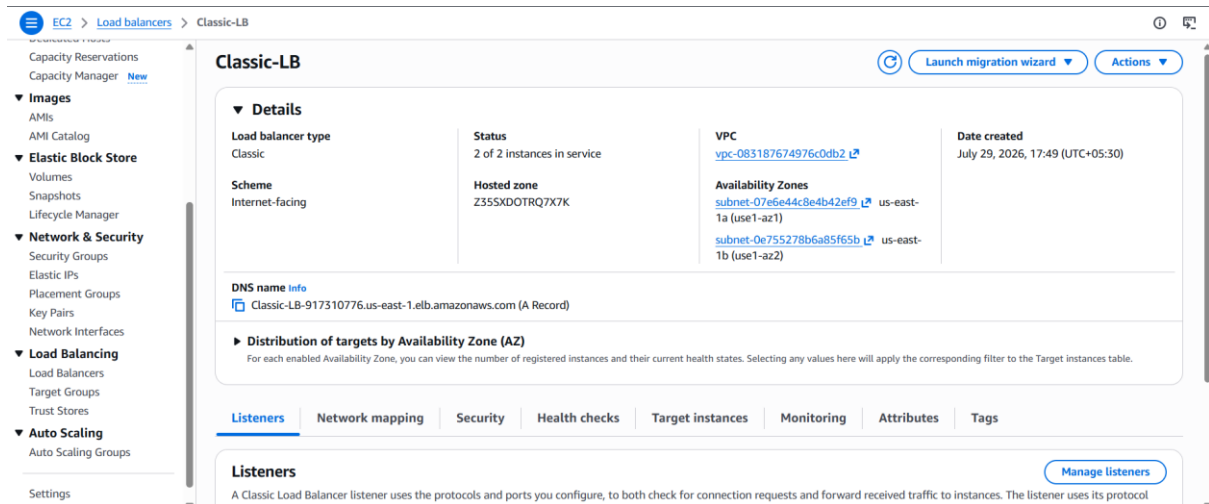
↓

Load Balancers



Instances

Status should be: InService



2. Configure Application Load balancer.

Objective: Configure an Application Load Balancer to distribute incoming HTTP traffic across multiple EC2 instances running Apache web servers.

Prerequisites

Before starting, make sure:

- ☒ Two EC2 instances are running.
- ☒ Apache (httpd) is installed on both instances.
- ☒ Both instances are in the **same VPC**.
- ☒ Security Groups allow **HTTP (80)**.
- ☒ Both instances are in **public subnets** (for an internet-facing ALB).

Step 1: Create Target Group

Go to

EC2 → Target Groups

Click

Create target group

Basic configuration

- Target Type → **Instances**
- Target Group Name → **Apache-TargetGroup**
- Protocol → **HTTP**
- Port → **80**
- VPC → Select your VPC

Click

Next

Step 2: Register Targets

Select

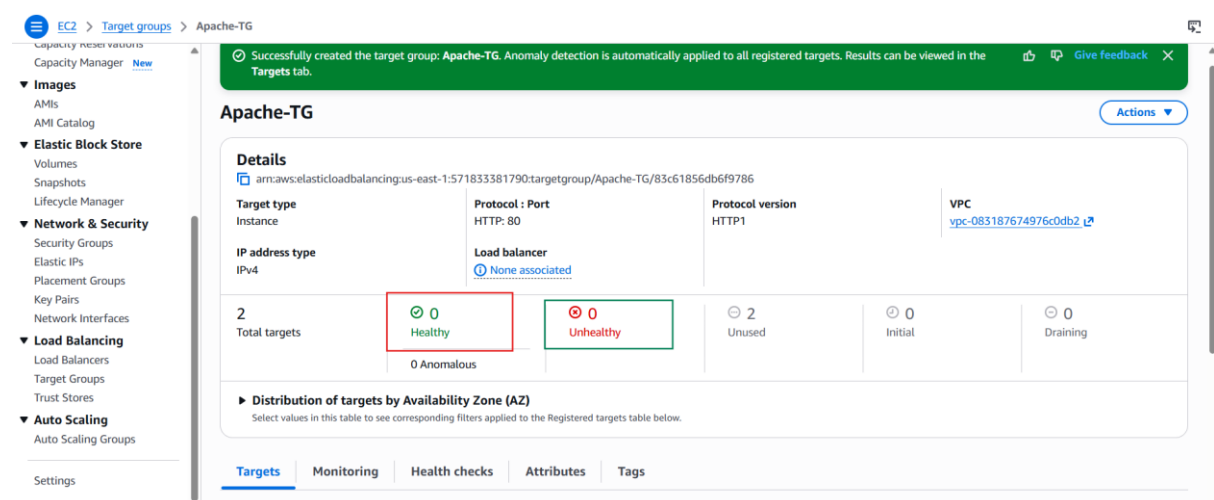
- Server-1
- Server-2

Click

Include as pending below

Then click

Create Target Group



EC2 > Target groups > Apache-TG

Capacity reserves resources
Capacity Manager [New](#)

▼ Images
AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

▼ Load Balancing
Load Balancers
Target Groups
Trust Stores

▼ Auto Scaling
Auto Scaling Groups

Settings

Successfully created the target group: Apache-TG. Anomaly detection is automatically applied to all registered targets. Results can be viewed in the Targets tab. [Give feedback](#)

Apache-TG

Actions

Details

arn:aws:elasticloadbalancing:us-east-1:571833381790:targetgroup/Apache-TG/83c61856db6f9786

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-083187674976c0db2
IP address type IPv4	Load balancer None associated		

2 Total targets	0 Healthy	0 Unhealthy	2 Unused	0 Initial	0 Draining
--------------------	--------------	----------------	-------------	--------------	---------------

0 Anomalous

► Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets Monitoring Health checks Attributes Tags

Step 5: Create Application Load Balancer

Go to

EC2 → Load Balancers

Click

Create Load Balancer

Choose

Application Load Balancer

Step 6: Configure Basic Settings

Name

Apache-ALB

Scheme

Internet-facing

IP Address Type

IPv4

Step 7: Network Mapping

Select

Your VPC

Choose at least **two public subnets** in different Availability Zones.

Example

Subnet-1 (us-east-1a)

Subnet-2 (us-east-1b)

Step 8: Security Group

Create a new security group or select an existing one.

Inbound Rules

Type Port Source

HTTP 80 0.0.0.0/0

Click

Next

Step 9: Configure Listener

Listener

HTTP

Port 80

Default Action

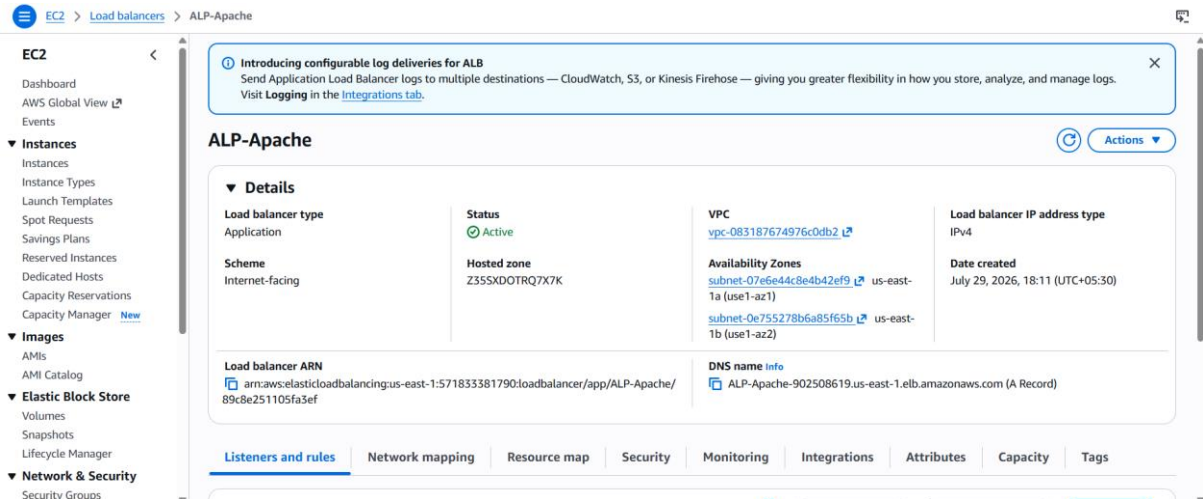
Select

Forward to

Apache-TG

Click

Create Load Balancer



Step 11: Test ALB

Open

EC2 → Load Balancers

Copy the **DNS name**.

Example

`http://Apache-ALB-123456789.us-east-1.elb.amazonaws.com`

Open it in your browser.

Refresh multiple times.

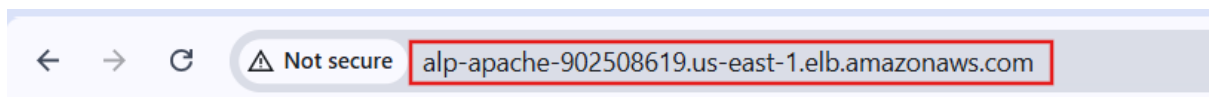
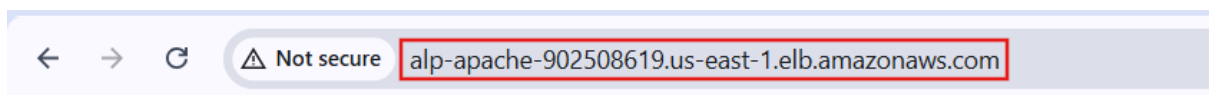
You should see:

Welcome to server-1

and

Welcome to Server 2

alternating between requests.



3. Configure Network Load balancer.

Objective: Configure an AWS Network Load Balancer (NLB) to distribute TCP traffic across multiple EC2 instances running an Apache web server.

Prerequisites

Before starting, ensure:

- Two EC2 instances are running.
- Apache (httpd) is installed and running on both instances.
- Both instances are in the same VPC.
- Both instances are in different Availability Zones (recommended).

- Port **80** is open in the EC2 security group.

Step 1: Create a Target Group

1. Go to **EC2 Console**.
2. Select **Target Groups**.
3. Click **Create target group**.

Configure:

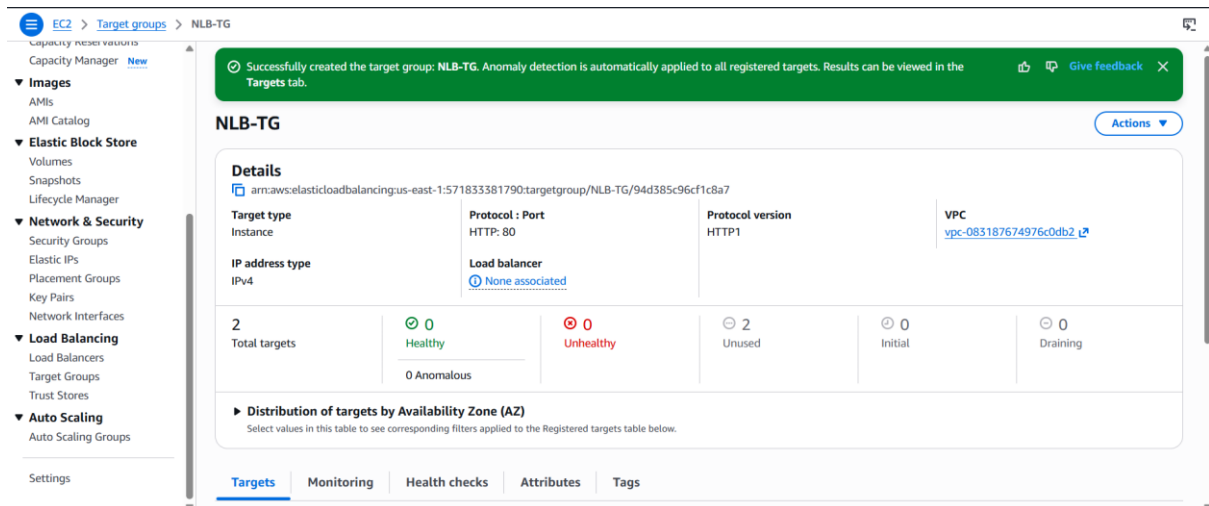
Setting	Value
Target Type	Instances
Name	NLB-TargetGroup
Protocol	TCP
Port	80
VPC	Select your VPC

Click **Next**.

Step 3: Register Targets

- Select **Server-1**
- Select **Server-2**
- Click **Include as pending below**
- Click **Create Target Group**

Wait until both targets become **Healthy**



Step 4: Create the Network Load Balancer

Go to:

EC2 → Load Balancers → Create Load Balancer

Choose:

Network Load Balancer

Click **Create**.

Step 5: Configure Basic Settings

Setting	Value
Name	Apache-NLB
Scheme	Internet-facing
IP address type	IPv4

Step 6: Network Mapping

Select:

- Your VPC
- Two public subnets

Example:

- us-east-1a
- us-east-1b

Click **Next**.

Step 7: Configure Listener

Setting Value

Protocol TCP

Port 80

Default action:

Forward to:

NLB-TargetGroup

Click **Next**.

Step 8: Select Target Group

Choose:

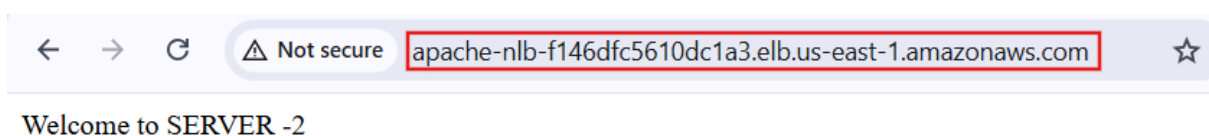
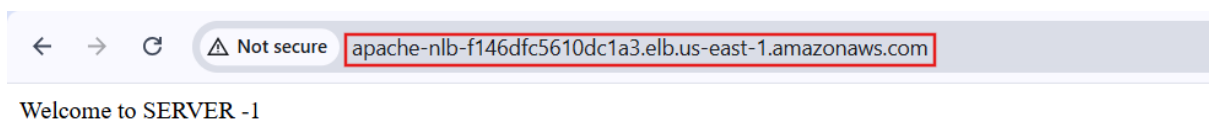
NLB-TargetGroup

Click:

Create Load Balancer

Wait until the status changes to:

Active



4. Attach SSL for application load balancer.

Objective: Configure **HTTPS (443)** on an Application Load Balancer by attaching an SSL/TLS certificate from **AWS Certificate Manager (ACM)**.

After configuration:

- Users can access the application using **HTTPS**
- ALB terminates SSL encryption
- ALB forwards traffic to EC2 instances using HTTP (Port 80)

Prerequisites

Before attaching SSL:

- ✓ Application Load Balancer created
- ✓ Target Group created
- ✓ EC2 instances healthy
- ✓ ACM certificate should be in **Issued** status
- ✓ Domain name should be validated in ACM

Step 1: Verify ACM Certificate

Go to:

AWS Console



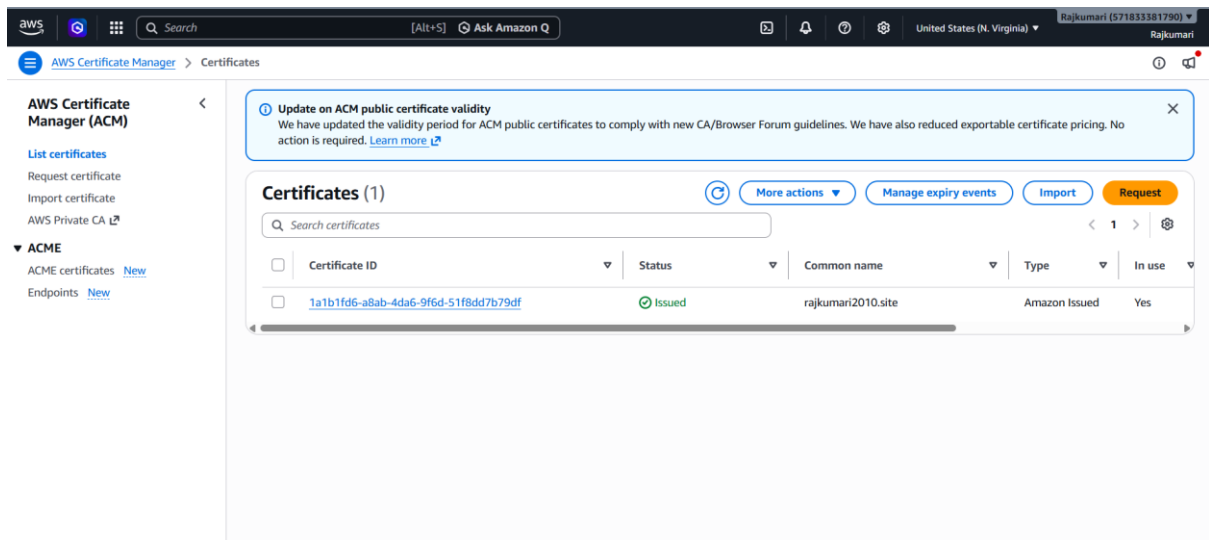
Certificate Manager (ACM)



Certificates

Check your certificate status:

Status: Issued



Step 2: Open Your Application Load Balancer

Go to:

EC2



Load Balancers



Select ALP-Apache

Step 3: Add HTTPS Listener

Open:

Listeners and rules

Click:

Add listener

Configure:

Protocol

HTTPS

Port: 443

Step 4: Select SSL Certificate

Under:

Secure listener settings

Choose:

Certificate source:

From ACM

Select your certificate:

Example:

rajkumari2010.site

Status: Issued

Step 5: Configure Default Action

For Default actions:

Select:

Forward to

Choose your target group:

Example:

Apache-TG

Step 6: Create Listener

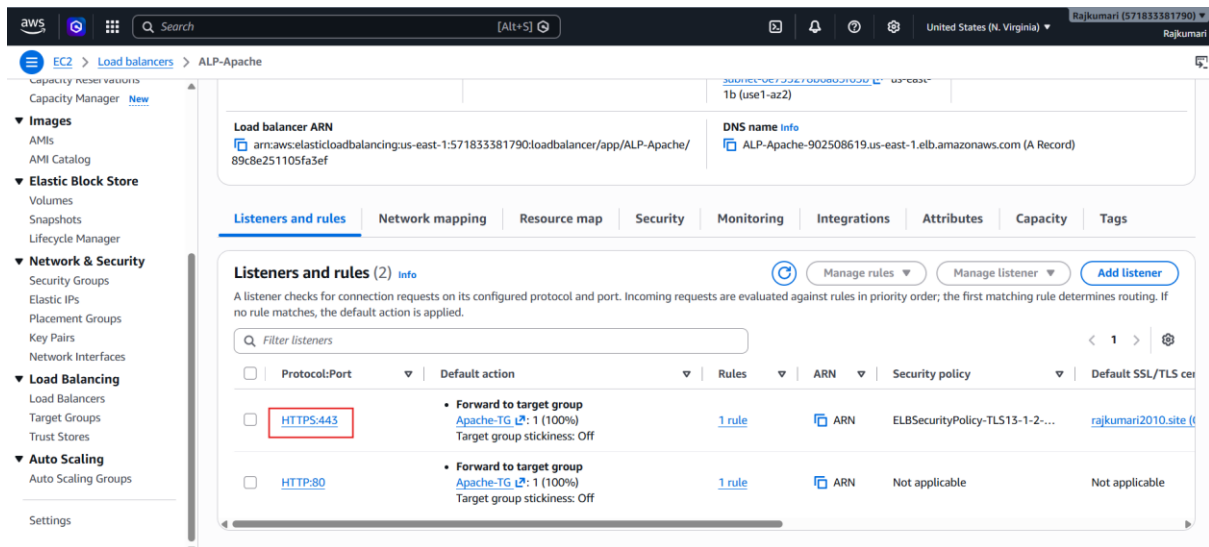
Click:

Add listener

Now your ALB has:

HTTP : 80

HTTPS : 443



Step 7: Update Security Group

Your ALB security group must allow HTTPS.

Go to:

EC2



Security Groups



Load Balancer Security Group



Inbound Rules

Add:

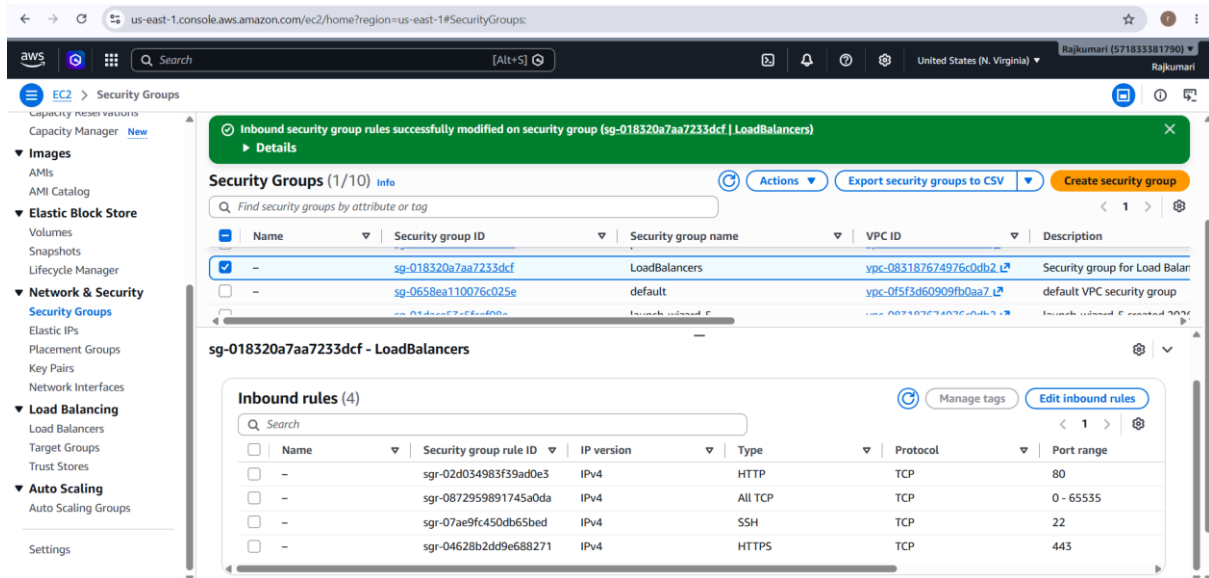
Type Port Source

HTTPS 443 0.0.0.0/0

Keep HTTP also if you want:

Type Port Source

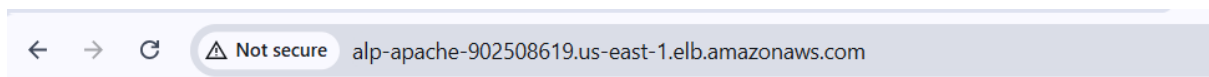
HTTP 80 0.0.0.0/0



Step 5: Test ALB again

After both targets become Healthy:

Open: <http://alp-apache-902508619.us-east-1.elb.amazonaws.com/>



Welcome to SERVER -1



Welcome to SERVER -2

5.Map Application load balancer to R53.

Objective: Map your **Application Load Balancer DNS name** with your custom domain name using **Amazon Route 53**.

Prerequisites

Before mapping:

- ✓ Application Load Balancer is created
- ✓ ALB status is **Active**
- ✓ Target Group targets are **Healthy**
- ✓ Domain is registered (for example, Namecheap domain)
- ✓ Route 53 Hosted Zone is created for your domain

Step 1: Open Route 53

Go to:

AWS Console



Route 53



Hosted zones

Step 2: Select Your Domain

Example:

rajkumari2010.site

Click the hosted zone.

You should see existing records:

Example:

Record name	Type
-------------	------

rajkumari2010.site	NS
--------------------	----

rajkumari2010.site	SOA
--------------------	-----

Step 3: Create an A Record (Alias)

Click:

Create record

Configure:

Record name

Leave empty for root domain:

rajkumari2010.site

or enter:

www

for:

www.rajkumari2010.site

Record type

Select:

A - IPv4 address

Alias

Enable:

Alias: ON

Route traffic to

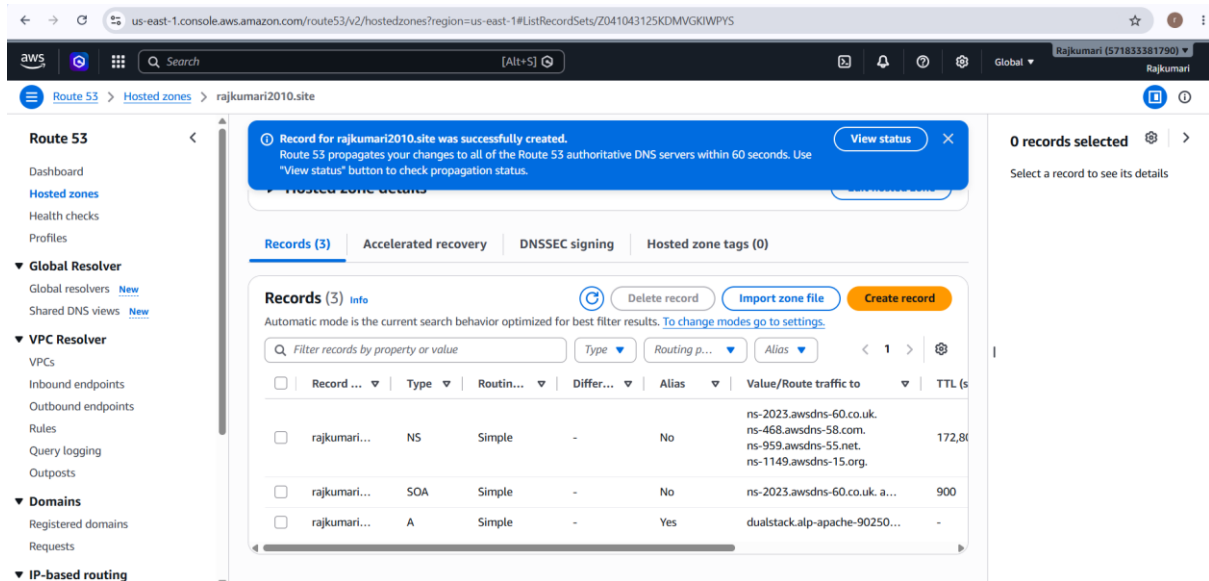
Select:

Alias to Application and Classic Load Balancer

Region

Select:

US East (N. Virginia) us-east-1



Step 4: Verify DNS Resolution

Wait a few minutes.

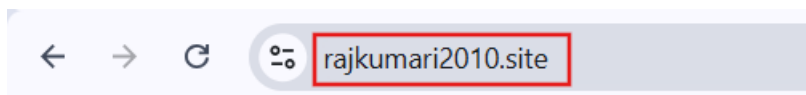
Example: rajkumari2010.site

You should see:

Welcome to SERVER-1

or

Welcome to SERVER-2



Welcome to SERVER -1



Welcome to SERVER -2

6. Push the application load balancer logs to S3.

Objective: Configure **Application Load Balancer access logs** to store request information in an **Amazon S3 bucket**.

ALB access logs contain details like:

- Client IP address
- Request time
- URL path
- HTTP status code
- Response time
- Target EC2 instance information

Step 1: Create an S3 Bucket for ALB Logs

Go to:

AWS Console

→ S3

→ Buckets

→ Create bucket

Example bucket name:

alp-apache-access-logs-2026

Choose the same region as your ALB:

us-east-1 (N. Virginia)

Keep:

- Block Public Access: ☒ Enabled
- Bucket Versioning: Optional

Click:

Create bucket

Step 2: Add Bucket Policy for ALB Access

ALB needs permission to write logs into S3.

Open:

S3

→ Your Bucket

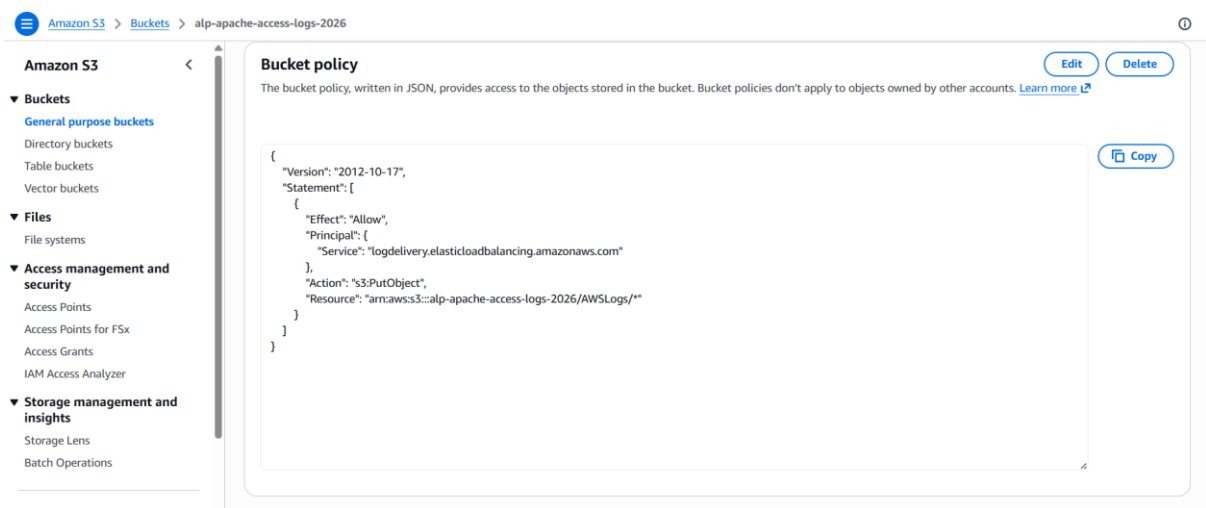
→ Permissions

→ Bucket Policy

Add this policy:

Replace:

YOUR-BUCKET-NAME



Step 3: Enable Access Logs on ALB

Go to:

EC2

→ Load Balancers

→ Select ALP-Apache

Go to:

Attributes

Find:

Access logs

Click:

Edit

Enable:

Access logs: ON

Step 4: Select S3 Bucket

Choose:

S3 location

Enter:

s3://alp-apache-access-logs-2026

Example:

Bucket:

alp-apache-access-logs-2026

Prefix (optional):

alb-logs

Click:

Save changes

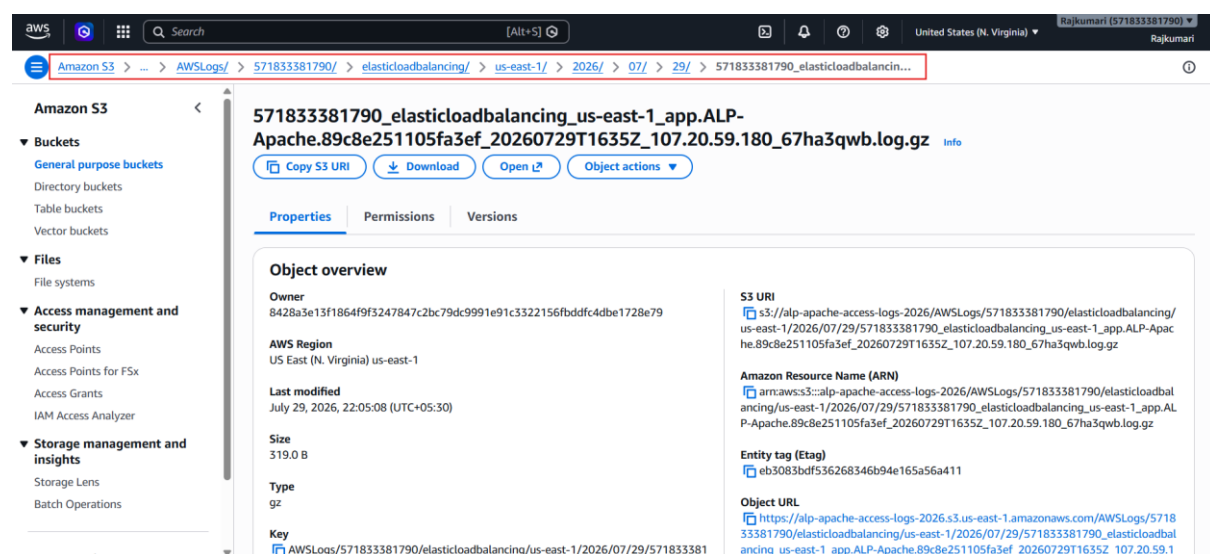
Step 5: Generate Traffic

Access your ALB:

<http://alp-apache-902508619.us-east-1.elb.amazonaws.com>

Refresh the page several times.

ALB will generate log files.



Step 6: Verify Logs in S3

Go to:

S3

→ Bucket

→ AWSLogs

You will see a structure like:

AWSLogs/

└─ Account-ID/

— elasticloadbalancing/

└─ us-east-1/

2026/

└─ ALB logs

Log files look like:

app.alp-apache.xxxxxxxx.log.gz

```
https 2026-07-29T16:33:36.385069Z app/ALP-Apache/89c8e251105fa3ef 206.105.81.88:5596 - 1 -1 -1 400 - 114 378 "GET http://107.20.59.180:443/ HTTP/1.1" "Mozilla/5.0 (zgrab/0.x)"
- 2026-07-29T16:33:36.385000Z - - - - - TID 3f9eb716475544e462df082a29b32 - 107.20.59.180
https 2026-07-29T16:33:36.499043Z app/ALP-Apache/89c8e251105fa3ef 206.105.81.88:55962 - 1 -1 -1 400 0 272 - http://alp-apache-902588619-us-east-1.elb.amazonaws.com:443 - - - - -
- 2026-07-29T16:33:36.455000Z - - - - - TID 91db371eb9379d47089ce29b6db4781 - 107.20.59.180
```